

Univerzitet u Novom Sadu
Fakultet tehničkih nauka
Departman za industrijsko inženjerstvo i menadžment
Predmet: Sistemi baza podataka



Podšema Rezervacija kursa

-projektni zadatak-

Predmetni profesor:

Andrija Manić IT5/2023

Danilo Nikolić

Novi Sad, 2025/26

Sadržaj:

1. Opis realnog sistema	4
2. Prikaz konceptualne šeme cele baze podataka	4
3. Prikaz odabrane podšeme uz njen opis	6
4. Tabelarni prikaz obeležja i ograničenja za odabranu podšemu	8
5. Relacioni model za odabranu podšemu	11
6. DDL naredbe za generisanje tabela i ograničenja	12
7. DML naredbe za unos podataka u tabele	16
8. SQL upiti sa kombinovanjem podataka iz više tabela, upotrebom agregatnih funkcija, podupita i slično	19
9. Sekvence sa primerom njihove upotrebe	22
10. Index	22
11. Procedure	23
12. Funkcija	25
13. Trigeri.....	26
14. Upotreba kursora u nekom od objekata	28
15. Upotreba transakcije.....	29
16. Zaključak	30

Sadržaj slika:

Slika 1. EER dijagram	4
Slika 2. Podšema Rezervacija kursa	6
Slika 3. DDL naredba za tabelu Kandidat.....	12
Slika 4. DDL naredbe za tabelu Nivo	12
Slika 5. DDL naredbe za tabelu Intenzitet	13
Slika 6. DDL naredbe za tabelu Jezik	13
Slika 7. DDL naredbe za tabelu Radi	14
Slika 8. DDL naredbe za tabelu Radi	14
Slika 9. DDL naredbe za tabelu Kurs	15
Slika 10. DDL naredbe za tabelu Rezervacija	15
Slika 11. DML naredbe za tabelu NIVO	16
Slika 12. DML naredbe za tabelu INTENZITET.....	16

Slika 13. DML naredbe za tabelu JEZIK	16
Slika 14. DML naredbe za tabelu KANDIDAT	17
Slika 15. DML naredbe za tabelu ULAZNI TEST.....	17
Slika 16. DML naredbe za tabelu RADI.....	17
Slika 17. DML naredbe za tabelu KURS.....	17
Slika 18. DML naredbe za tabelu REZERVACIJA.....	18
Slika 19. SQL upit 1	19
Slika 20. SQL upit 2	19
Slika 21. SQL upit 3	20
Slika 22. SQL upit 4	20
Slika 23. SQL upit 5	21
Slika 24. Sekvenca za Rezervaciju	22
Slika 25. Sekvenca za Kandidata	22
Slika 26. Index.....	22
Slika 27. Procedura EvidentirajTestIDodeliKurs.....	23
Slika 28. Procedura OtkaziRezervacijuNeaktivnihKurseva	24
Slika 29. Funkcija NaprednaAnalizaKandidata.....	25
Slika 30. Triger ObrisiZastareluRezervaciju	26
Slika 31. Triger ValidacijaRezervacije	27
Slika 32. Kursor 1	Error! Bookmark not defined.
Slika 33. Transakcija	29

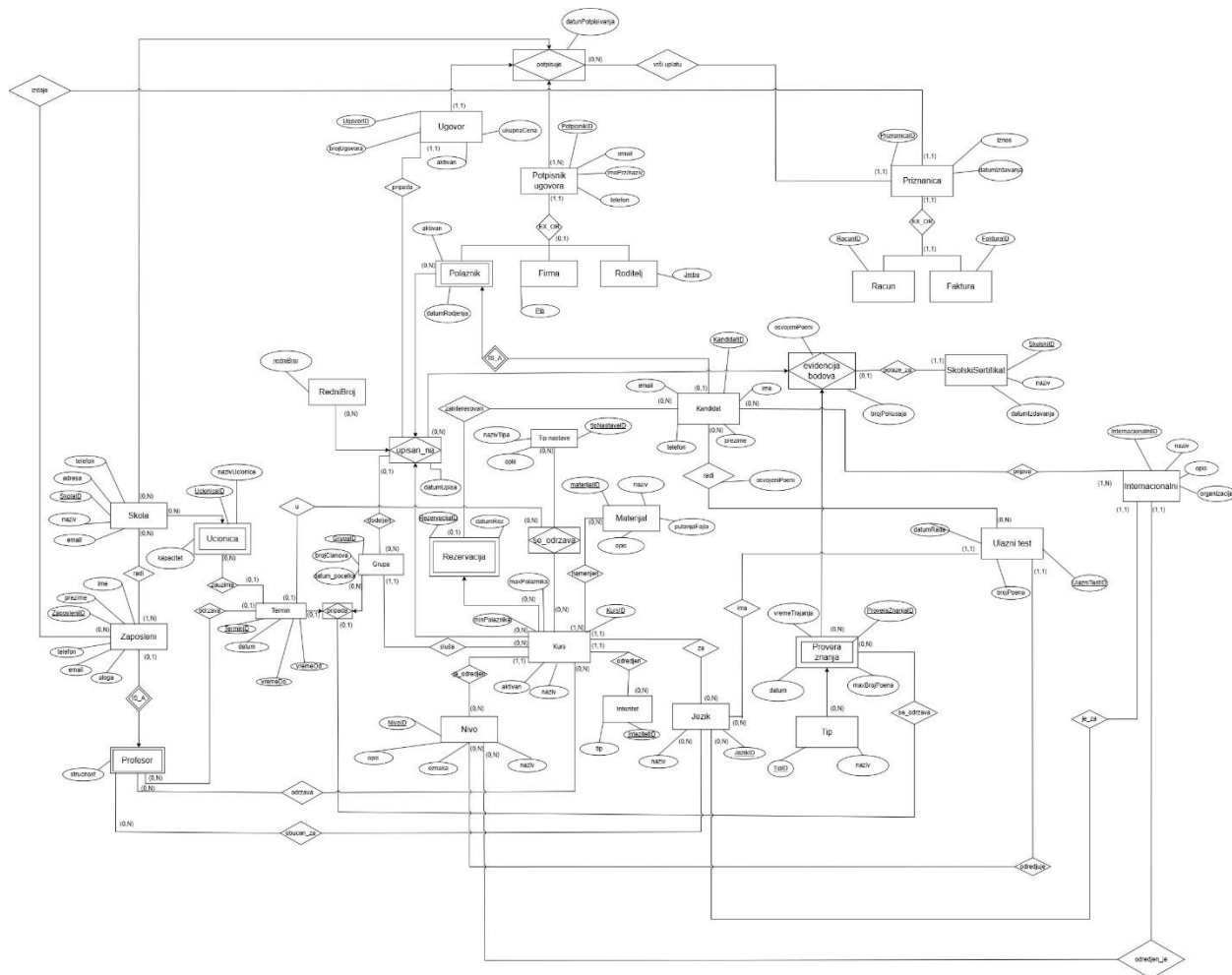
Sadržaj tabela:

Tabela 1. Tip poveznika RADI	8
Tabela 2. Tip entiteta KURS	8
Tabela 3. Tip entiteta NIVO.....	8
Tabela 4. Tip entiteta INTENZITET	9
Tabela 5. Tip entiteta JEZIK.....	9
Tabela 6. Tip entiteta ULAZNI TEST	9
Tabela 7. Tip entiteta KANDIDAT	10
Tabela 8. Tip entiteta REZERVACIJA.....	10

1. Opis realnog sistema

Domen projekta predstavlja poslovanje škole stranih jezika čija je osnovna delatnost organizovanje i realizacija kurseva stranih jezika, koji podrazumevaju samointenzitet časova različitih jezika, različite nivoe i stepen dostupnih jezika, kao i profesore koji predaju. Poslovni procesi koji se pominju unutar ovog sistema uključuju upis polaznika, formiranje grupa prema uzrastu i nivou znanja na prijemnom testu, realizaciju nastave, testiranje stečenog znanja, kao i administrativne i finansijske aktivnosti koje obavljaju zaposleni sistema. Škola posluje kao preduzeće sa fleksibilnim raspodelama zaduženja zaposlenih jer sistem funkcioniše tako da svaki zaposleni može da obavlja mnoge različite funkcije unutar preduzeća i nije ograničen na samo jedan posao.

2. Prikaz konceptualne šeme cele baze podataka



Slika 1. EER dijagram

Opis EER dijagrama:

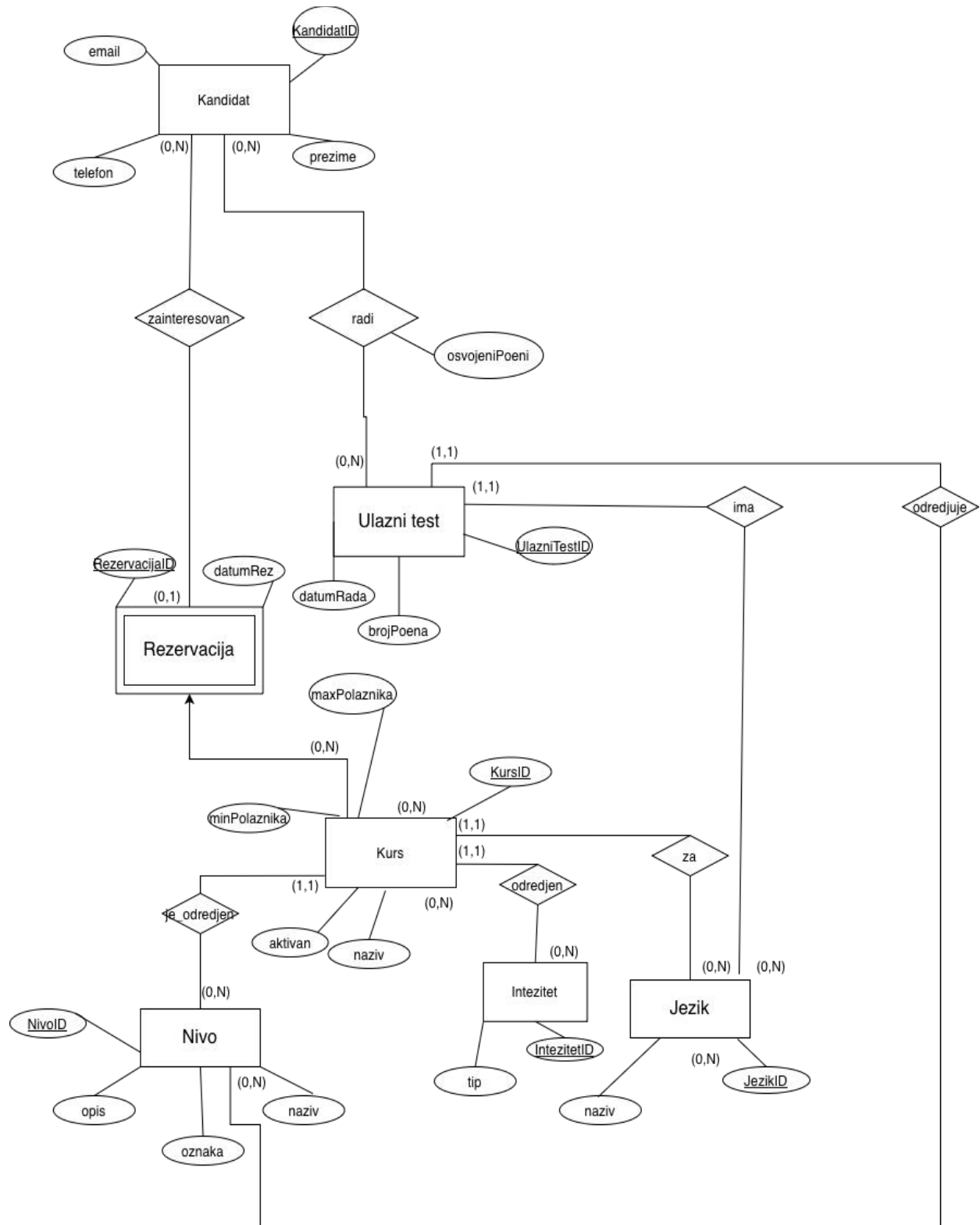
Konceptualna šema informacionog sistema škole stranih jezika modelovana je pomoću EER dijagrama kako bi se prikazali svi važni delovi sistema i njihove međusobne veze. Centralni deo sistema predstavlja entitet Kurs, jer škola svoju osnovnu delatnost zasniva na organizaciji i realizaciji kurseva stranih jezika. Svaki kurs povezan je sa određenim jezikom, nivoom znanja i tipom kursa, čime je omogućena jasna organizacija nastave.

Na osnovu kurseva formiraju se grupe polaznika koje su povezane sa profesorima, terminima i učionicama u kojima se nastava održava. U sistemu se evidentiraju podaci o polaznicima, njihovim upisima i rezervacijama u slučaju da grupa nema slobodnih mesta ili kurs trenutno nije aktivan.

Model obuhvata i zaposlene u školi, pri čemu jedan zaposleni može imati više različitih zaduženja. Poseban deo sistema odnosi se na ugovore, uplate i izdavanje računa i faktura, kako bi se omogućilo praćenje poslovanja škole.

Tokom trajanja kursa evidentiraju se različite provere znanja i rezultati polaznika, kao i ulazni testovi koji služe za određivanje nivoa znanja pre početka pohađanja kursa. Na kraju kursa čuvaju se podaci o postignućima i uspešnosti polaznika. Na ovaj način EER dijagram omogućava jasan prikaz kompletne organizacije rada škole stranih jezika i predstavlja osnovu za dalje projektovanje baze podataka.

3. Prikaz odabrane podšeme uz njen opis



Slika 2. Podšema Rezervacija kusa

Opis podšeme REZERVACIJA KURSA:

Tip entiteta KANDIDAT sadrži osnovne podatke o kandidatu: KandidatID, ime, prezime, telefon i email. Kandidat može da polaže ulazni test, ali i da iskaže interesovanje za određeni kurs. Tip entiteta ULAZNI TEST sadrži podatke o testu: UlazniTestID, datumRada i brojPoena. ULAZNI TEST je povezan sa tipom enteta JEZIK kako bi imali uvid na koji JEZIK se odnosi ULAZNI TEST. Kandidat i ulazni test povezani su tipom poveznika RADI, koji predstavlja čin polaganja testa. Ovaj tip poveznika ima obeležje osvojeniPoeni, koji pokazuje koliko je poena kandidat ostvario na konkretnom testu. Na osnovu rezultata testa određuje se odgovarajući kurs za kandidata. To je predstavljeno tipom poveznika ODREĐUJE, koji povezuje ulazni test sa tipom entitetom NIVO. Na taj način test usmerava kandidata na kurs koji odgovara njegovom znanju. Pored toga, kandidat može biti zainteresovan za određeni kurs i pre polaganja testa, što je modelovano tipom poveznika ZAINTERESOVAN između tipova entiteta KANDIDAT i KURS. Tip entiteta KURS sadrži podatke: KursID, naziv, aktivan, minPolaznika i maxPolaznika. Svaki kurs je definisan određenim nivoom, intenzitetom i jezikom. Tip entiteta NIVO sadrži NivoID, naziv, oznaku i opis, a kurs je sa nivoom povezan vezom koja određuje kom nivou pripada. Tip entiteta INTENZITET sadrži IntenzitetID, naziv i tip i definiše dinamiku održavanja kursa (npr. standardni, intenzivni). Tip entiteta JEZIK sadrži JezikID i naziv, a kurs je povezan sa jezikom preko odgovarajuće veze, čime se definiše za koji jezik je kurs organizovan. Tip entiteta REZERVACIJA predstavlja rezervaciju mesta na budućem kursu i sadrži RezervacijaID i datumRez. Rezervacija omogućava kandidatu da obezbedi mesto na kursu pre njegovog početka.

4. Tabelarni prikaz obeležja i ograničenja za odabranu podšemu

Tabela 1. Tip poveznika *RADI*

Redni broj	Naziv obeležja	Opis obeležja	Tip podatka	Ograničenje	Null
1	osvojenPoeni	Broj osvojenih poena na testu	INT	Mora biti od 0 do 100	⊥

Tabela 2. Tip entiteta *KURS*

Redni broj	Naziv obeležja	Opis obeležja	Tip podatka	Ograničenje	Null
1	kursID	Identifikacioni broj kursa	INT	Primarni ključ	⊥
2	aktivan	Da li je kurs trenutno aktivan	BIT	/	⊥
3	minPolaznika	Minimalan broj polaznika na kursu	INT	Mora biti najmanje 1	⊥
4	maxPolaznika	Maksimalan broj polaznika na kursu	INT	Može biti najviše 60	⊥
5	naziv	Naziv kursa	VARCHAR(25)	/	T

Tabela 3. Tip entiteta *NIVO*

Redni broj	Naziv obeležja	Opis obeležja	Tip podatka	Ograničenje	Null
1	nivoID	Identifikacioni broj nivoa	INT	Primarni ključ	⊥
2	opis	Opis nivoa	VARCHAR(20)	/	⊥
3	oznaka	Oznaka nivoa	VARCHAR(20)	Mora biti A1, A2, B1, B2, C1, C2	⊥
4	naziv	Naziv nivoa	VARCHAR(20)	/	T

Tabela 4. Tip entiteta INTENZITET

Redni broj	Naziv obeležja	Opis obeležja	Tip podatka	Ograničenje	Null
1	intezitetID	Identifikacioni broj inteziteta	INT	Primarni ključ	⊥
2	tip	Tip inteziteta	VARCHAR(20)	Mora biti osnovni, srednji, intenzivni	⊥

Tabela 5. Tip entiteta JEZIK

Redni broj	Naziv obeležja	Opis obeležja	Tip podatka	Ograničenje	Null
1	jezikID	Identifikacioni broj jezika	INT	Primarni ključ	⊥
2	naziv	Naziv jezika	VARCHAR(20)	/	⊥

Tabela 6. Tip entiteta ULAZNI TEST

Redni broj	Naziv obeležja	Opis obeležja	Tip podatka	Ograničenje	Uslov
1	ulazniTestID	Identifikacioni broj ulaznog testa	INT	Primarni ključ	⊥
2	datumRada	Datum rada testa	DATE	Ako se ne unese, DEFAULT treba da bude datum unosa	⊥
3	brojPoena	Broj poena koje je moguće ostvariti na testu	INT	Mora biti od 0 do 100	⊥

Tabela 7. Tip entiteta KANDIDAT

Redni broj	Naziv obeležja	Opis obeležja	Tip podatka	Ograničenje	Null
1	kandidatID	Identifikacion i broj kandidata	INT	Primarni ključ	⊥
2	ime	Ime kandidata	VARCHAR(20)	/	⊥
3	prezime	Prezime kandidata	VARCHAR(20)	/	⊥
4	email	Email kandidata	VARCHAR(20)	Mora biti jedinstven	T
5	telefon	Broj telefona kandidata	VARCHAR(20)	Mora biti jedinstven	T
Ključ	K={KandidatID}				

Tabela 8. Tip entiteta REZERVACIJA

Redni broj	Naziv obeležja	Opis obeležja	Tip podatka	Ograničenje	Uslov
1	RezervacijaID	Identifikacioni broj rezervacije	INT	Primarni ključ	⊥
2	datumRaz	Datum rezervacije kursa	DATE	Ako se ne unese, DEFAULT treba da bude datum unosa	⊥

5. Relacioni model za odabranu podšemu

Nivo({nivoID, naziv, oznaka, opis}, {nivoID})

Intenzitet({intenzitetID, tip}, {intenzitetID})

Jezik({jezikID, naziv}, {jezikID})

Kandidat({kandidatID, ime, prezime, telefon, email}, {kandidatID})

UlazniTest({ulazniTestID, brojPoena, datumRada, nivoID, jezikID}, {ulazniTestID})

UlazniTest[jezikID] $\subseteq$ Jezik[jezikID]

UlazniTest[nivoID] $\subseteq$ Nivo[nivoID]

Null(UlazniTest, jezikID) = $\perp$

Null(UlazniTest, nivoID) = $\perp$

Radi({kandidatID, ulazniTestID, osvojeniPoeni}, {kandidatID, ulazniTestID})

Radi[kandidatID] $\subseteq$ Kandidat[kandidatID]

Radi[ulazniTestID] $\subseteq$ UlazniTest[ulazniTestID]

Kurs({kursID, minPolaznika, maxPolaznika, aktivan, naziv, nivoID, intenzitetID, jezikID}, {kursID})

Kurs[nivoID] $\subseteq$ Nivo[nivoID]

Kurs[jezikID] $\subseteq$ Jezik[jezikID]

Kurs[intenzitetID] $\subseteq$ Intenzitet[intenzitetID]

Null(Kurs, nivoID) = $\perp$

Null(Kurs, jezikID) = $\perp$

Null(Kurs, intenzitetID) = $\perp$

Rezervacija({rezervacijaID, datumRez, kursID, kandidatID}, {rezervacijaID, kursID})

Rezervacija[kandidatID] $\subseteq$ Kandidat[kandidatID]

Rezervacija[kursID] $\subseteq$ Kurs[kursID]

Null(Rezervacija, kursID) = $\perp$

Null(Rezervacija, kandidatID) = $\perp$

6. DDL naredbe za generisanje tabela i ograničenja

Tabela KANDIDAT:

Na slici je prikazana DDL naredba za kreiranje tabele Kandidat u okviru podšeme sistema za rezervaciju kurseva. U tabeli se čuvaju osnovni podaci o kandidatima kao što su ime, prezime, email i broj telefona. Primarni ključ tabele je atribut kandidatID koji služi za jedinstvenu identifikaciju kandidata, dok su atributi email i telefon definisani kao jedinstveni kako bi se sprečilo unošenje duplih podataka. Ova tabela koristi se za evidenciju kandidata zainteresovanih za pohađanje kurseva stranih jezika.

```
CREATE TABLE RezervacijaKursa.Kandidat(  
kandidatID INT NOT NULL PRIMARY KEY DEFAULT (NEXT VALUE FOR RezervacijaKursa.KandidatSeq),  
ime VARCHAR(20) NOT NULL,  
prezime VARCHAR(20) NOT NULL,  
email VARCHAR(25) NULL UNIQUE,  
telefon VARCHAR(12) NULL UNIQUE  
);  
go
```

Slika 3. DDL naredba za tabelu Kandidat

Tabela NIVO:

Na slici je prikazana DDL naredba za kreiranje tabele Nivo u okviru podšeme sistema za rezervaciju kurseva. U tabeli se čuvaju podaci o nivoima znanja koji su dostupni u školi, uključujući naziv, oznaku i opis nivoa. Primarni ključ tabele je atribut nivoID koji služi za jedinstvenu identifikaciju svakog nivoa i koristi IDENTITY(1,1) za automatsko generisanje vrednosti. Tabela sadrži i ograničenje čeka (CHECK constraint) pod nazivom CK_Nivo_oznaka koje osigurava da uneta oznaka nivoa mora biti isključivo jedna od standardnih vrednosti: A1, A2, B1, B2, C1 ili C2.

```
CREATE TABLE RezervacijaKursa.Nivo(  
nivoID INT NOT NULL PRIMARY KEY IDENTITY(1,1),  
naziv VARCHAR(20) NULL,  
oznaka VARCHAR(20) NOT NULL,  
opis VARCHAR(20) NOT NULL,  
CONSTRAINT CK_Nivo_oznaka CHECK (oznaka IN ('A1', 'A2', 'B1', 'B2', 'C1', 'C2'))  
);  
go
```

Slika 4. DDL naredbe za tabelu Nivo

Tabela INTENZITET:

Na slici je prikazana DDL naredba za kreiranje tabele Intenzitet u okviru podšeme sistema za rezervaciju kurseva. U tabeli se čuvaju podaci o dinamici i intenzitetu održavanja nastave. Primarni ključ tabele je atribut intenzitetID koji služi za jedinstvenu identifikaciju svakog tipa intenziteta uz automatsko inkrementiranje. Tabela sadrži i ograničenje čeka (CHECK constraint) pod nazivom CK_Intenzitet_tip koje striktno ograničava unos atributa tip na vrednosti: 'Osnovni', 'Srednji' ili 'Intenzivni'.

```
CREATE TABLE RezervacijaKursa.Intenzitet(  
intenzitetID INT NOT NULL PRIMARY KEY IDENTITY(1,1),  
tip VARCHAR(20) NOT NULL,  
CONSTRAINT CK_Intenzitet_tip CHECK (tip IN ('Osnovni', 'Srednji', 'Intenzivni'))  
);  
go
```

Slika 5. DDL naredbe za tabelu Intenzitet

Tabela JEZIK:

Na slici je prikazana DDL naredba za kreiranje tabele Jezik u okviru podšeme sistema za rezervaciju kurseva. U tabeli se čuvaju podaci o stranim jezicima koji se predaju u školi. Primarni ključ tabele je atribut jezikID koji obezbeđuje jedinstvenost svakog zapisa i automatski se uvećava. Pored ključa, tabela sadrži atribut naziv koji je definisan kao obavezan unos (NOT NULL) kako bi se osiguralo da svaki uneti jezik ima svoj naziv.

```
CREATE TABLE RezervacijaKursa.Jezik(  
jezikID INT NOT NULL PRIMARY KEY IDENTITY(1,1),  
naziv VARCHAR(20) NOT NULL  
);  
go
```

Slika 6. DDL naredbe za tabelu Jezik

Tabela ULAZNI TEST:

Na slici je prikazana DDL naredba za kreiranje tabele **UlazniTest** u okviru podšeme sistema za rezervaciju kurseva. U tabeli se vode podaci o testovima koje kandidati rade radi provere znanja, a sadrži attribute brojPoena, datumRada, kao i spoljne ključeve nivelID i jezikID. Primarni ključ je ulazniTestID, dok atribut datumRada ima podrazumevanu vrednost (DEFAULT GETDATE()) koja automatski beleži trenutni datum ako se on eksplicitno ne unese. Tabela sadrži strana ključna ograničenja koja je povezuju sa tabelama Nivo i Jezik, kao i ček ograničenje CK_BrojPoeni koje dozvoljava unos poena isključivo u rasponu od 0 do 100.

```

CREATE TABLE RezervacijaKursa.UlazniTest(
ulazniTestID INT NOT NULL PRIMARY KEY IDENTITY(1,1),
brojPoena INT NOT NULL,
datumRada DATE NOT NULL DEFAULT GETDATE(),
nivoID INT NOT NULL,
jezikID INT NOT NULL,
CONSTRAINT FK_UlazniTest_Nivo FOREIGN KEY (nivoID) REFERENCES RezervacijaKursa.Nivo(nivoID),
CONSTRAINT FK_UlazniTest_Jezik FOREIGN KEY (jezikID) REFERENCES RezervacijaKursa.Jezik(jezikID),
CONSTRAINT CK_BrojPoeni CHECK (brojPoena BETWEEN 0 AND 100)
);
go

```

Slika 7. DDL naredbe za tabelu Radi

Tabela RADI:

Na slici je prikazana DDL naredba za kreiranje tabele Radi u okviru podšeme sistema za rezervaciju kurseva, koja predstavlja poveznik između kandidata i ulaznog testa. Tabela sadrži atribut osvojeniPoeni i identifikatore povezanih entiteta. Primarni ključ je složen i sastoji se od kombinacije atributa kandidatID i ulazniTestID. Tabela implementira dva strana ključa koji obezbeđuju referencijalni integritet prema tabelama Kandidat i UlazniTest, kao i ček ograničenje CK_OsvojeniPoeni koje garantuje da osvojeni bodovi moraju biti u opsegu od 0 do 100.

```

CREATE TABLE RezervacijaKursa.Radi(
osvojeniPoeni INT NOT NULL,
kandidatID INT NOT NULL,
ulazniTestID INT NOT NULL,
CONSTRAINT PK_Rad PRIMARY KEY (kandidatID, ulazniTestID),
CONSTRAINT FK_Rad_Kandidat FOREIGN KEY (kandidatID) REFERENCES RezervacijaKursa.Kandidat(kandidatID),
CONSTRAINT FK_Rad_UlazniTest FOREIGN KEY (ulazniTestID) REFERENCES RezervacijaKursa.UlazniTest(ulazniTestID),
CONSTRAINT CK_OsvojeniPoeni CHECK (osvojeniPoeni BETWEEN 0 AND 100)
);
go

```

Slika 8. DDL naredbe za tabelu Radi

Tabela KURS:

Na slici je prikazana DDL naredba za kreiranje tabele Kurs u okviru podšeme sistema za rezervaciju kurseva. U tabeli se čuvaju podaci o kursevima i njihovim kapacitetima kroz attribute aktivan i naziv, kao i atributieminPolaznika i maxPolaznika koji imaju svoja ograničenja. Primarni ključ je kursID sa automatskim generisanjem vrednosti. Tabela sadrži tri strana ključa (FOREIGN KEY) koji je povezuju sa tabelama Nivo, Intenzitet i Jezik, čime se obezbeđuje da svaki kurs bude precizno definisan u pogledu jezika, nivoa znanja i dinamike nastave.

```

CREATE TABLE RezervacijaKursa.Kurs(
kursID INT NOT NULL PRIMARY KEY IDENTITY(1,1),
aktivan BIT NOT NULL,
naziv VARCHAR(20) NULL,
minPolaznika INT NOT NULL,
maxPolaznika INT NOT NULL,
nivoID INT NOT NULL,
intenzitetID INT NOT NULL,
jezikID INT NOT NULL,
CONSTRAINT FK_Kurs_Nivo FOREIGN KEY (nivoID) REFERENCES RezervacijaKursa.Nivo(nivoID),
CONSTRAINT FK_Kurs_Intenzitet FOREIGN KEY (intenzitetID) REFERENCES RezervacijaKursa.Intenzitet(intenzitetID),
CONSTRAINT FK_Kurs_Jezik FOREIGN KEY (jezikID) REFERENCES RezervacijaKursa.Jezik(jezikID),
CONSTRAINT CK_Kurs_min CHECK (minPolaznika >= 1),
CONSTRAINT CK_Kurs_max CHECK (maxPolaznika <=60)
);
go

```

Slika 9. DDL naredbe za tabelu Kurs

Tabela REZERVACIJA:

Na slici je prikazana DDL naredba za kreiranje tabele Rezervacija u okviru podšeme sistema za rezervaciju kurseva. U tabeli se evidentiraju podaci o rezervisanim mestima na kursevima, a sadrži attribute rezervacijaID, datumRez, kursID i kandidatID. Primarni ključ je složen i čine ga rezervacijaID i kursID. Atribut datumRez koristi podrazumevanu vrednost (DEFAULT GETDATE()) za automatsko unošenje datuma, dok strana ključna ograničenja obezbeđuju čvrstu vezu i referencijalni integritet sa tabelama Kurs i Kandidat.

```

CREATE TABLE RezervacijaKursa.Rezervacija(
rezervacijaID INT NOT NULL DEFAULT (NEXT VALUE FOR RezervacijaKursa.RezervacijaSeq),
datumRez DATE NOT NULL DEFAULT GETDATE(),
kursID INT NOT NULL,
kandidatID INT NOT NULL,
CONSTRAINT PK_Rezervacija PRIMARY KEY (rezervacijaID, kursID),
CONSTRAINT FK_Rezervacija_Kurs FOREIGN KEY (kursID) REFERENCES RezervacijaKursa.Kurs(kursID),
CONSTRAINT FK_Rervacija_Kandidat FOREIGN KEY (kandidatID) REFERENCES RezervacijaKursa.Kandidat(kandidatID)
);
go

```

Slika 10. DDL naredbe za tabelu Rezervacija

7. DML naredbe za unos podataka u tabele

Nakon kreiranja strukture tabela i definisanja svih pravila i ograničenja u DDL delu, sledeći korak je inicijalno popunjavanje baze podacima pomoću INSERT INTO naredbi.

Uneti podaci su testnog karaktera i pažljivo su osmišljeni kako bi simulirali realne situacije iz svakodnevnog rada škole stranih jezika. Prilikom unosa strogo se vodilo računa o poštovanju svih prethodno postavljenih pravila – od primarnih ključeva i jedinstvenih vrednosti, preko relacionih veza (stranih ključeva), pa sve do CHECK i DEFAULT ograničenja. Na ovaj način je obezbeđen početni skup podataka koji služi kao stabilna osnova za proveru ispravnosti rada baze i izvršavanje kasnijih SQL upita.

U nastavku je prikazan kod za unos podataka za svaku pojedinačnu tabelu iz odabrane podšeme sistema.

Popunjavanje tabele NIVO:

```
INSERT INTO RezervacijaKursa.Nivo (naziv, oznaka, opis) VALUES
('Početni 1', 'A1', 'Nivo A1.1'),
('Početni 2', 'A1', 'Nivo A1.2'),
('Osnovni 1', 'A2', 'Nivo A2.1'),
('Osnovni 2', 'A2', 'Nivo A2.2'),
('Srednji 1', 'B1', 'Nivo B1.1'),
('Srednji 2', 'B1', 'Nivo B1.2'),
('Viši srednji 1', 'B2', 'Nivo B2.1'),
('Viši srednji 2', 'B2', 'Nivo B2.2'),
('Napredni 1', 'C1', 'Nivo C1.1'),
('Ekspertski 1', 'C2', 'Nivo C2.1');
```

Slika 11. DML naredbe za tabelu NIVO

Popunjavanje tabele INTENZITET:

```
INSERT INTO RezervacijaKursa.Intenzitet (tip) VALUES
('Osnovni'), ('Srednji'), ('Intenzivni'),
('Osnovni'), ('Srednji'), ('Intenzivni'),
('Osnovni'), ('Srednji'), ('Intenzivni'),
('Intenzivni');
```

Slika 12. DML naredbe za tabelu INTENZITET

Popunjavanje tabele JEZIK:

```
INSERT INTO RezervacijaKursa.Jezik (naziv) VALUES
('Engleski'), ('Nemački'), ('Francuski'), ('Italijanski'), ('Španski'),
('Ruski'), ('Norveški'), ('Švedski'), ('Kineski'), ('Japanski');
```

Slika 13. DML naredbe za tabelu JEZIK

Popunjavanje tabele KANDIDAT:

```
INSERT INTO RezervacijaKursa.Kandidat (ime, prezime, email, telefon) VALUES
('Marko', 'Marković', 'marko@mail.com', '064111222'),
('Ana', 'Anić', 'ana@mail.com', '064333444'),
('Petar', 'Petrović', 'petar@mail.com', '065555666'),
('Jelena', 'Jelić', 'jelena@mail.com', '061777888'),
('Nikola', 'Nikić', 'nikola@mail.com', '062999000'),
('Sara', 'Sarić', 'sara@mail.com', '063123456'),
('Luka', 'Lukić', 'luka@mail.com', '066456789'),
('Milica', 'Milić', 'milica@mail.com', '060111999'),
('Stefan', 'Stević', 'stefan@mail.com', '069222888'),
('Ivana', 'Ivić', 'ivana@mail.com', '067333777');
```

Slika 14. DML naredbe za tabelu KANDIDAT

Popunjavanje tabele ULAZNI TEST:

```
INSERT INTO RezervacijaKursa.UlazniTest (brojPoena, datumRada, nivoID, jezikID) VALUES
(85, '2026-05-01', 1, 1),
(45, '2026-05-02', 2, 2),
(92, '2026-05-03', 9, 1),
(70, '2026-05-04', 5, 3),
(30, '2026-05-05', 1, 5),
(100, '2026-05-06', 10, 1),
(55, '2026-05-07', 3, 2),
(88, '2026-05-08', 7, 1),
(62, '2026-05-09', 5, 4),
(75, '2026-05-10', 8, 2);
```

Slika 15. DML naredbe za tabelu ULAZNI TEST

Popunjavanje tabele RADI:

```
INSERT INTO RezervacijaKursa.Radi (osvojeniPoeni, kandidatID, ulazniTestID) VALUES
(85, 1, 1), (45, 2, 2), (92, 3, 3), (70, 4, 4), (30, 5, 5),
(100, 6, 6), (55, 7, 7), (88, 8, 8), (62, 9, 9), (75, 10, 10);
```

Slika 16. DML naredbe za tabelu RADI

Popunjavanje tabele KURS:

```
INSERT INTO RezervacijaKursa.Kurs (aktivan, naziv, minPolaznika, maxPolaznika, nivoID, intenzitetID, jezikID) VALUES
(1, 'Engleski A1', 5, 12, 1, 1, 1),
(1, 'Nemački A1', 4, 10, 2, 2, 2),
(1, 'Engleski C1', 3, 8, 9, 3, 1),
(0, 'Francuski B1', 5, 12, 5, 1, 3),
(1, 'Španski A1', 6, 15, 1, 1, 5),
(1, 'Ruski A2', 4, 10, 3, 2, 6),
(1, 'Engleski B2', 5, 12, 7, 3, 1),
(1, 'Italijanski B1', 4, 10, 5, 1, 4),
(1, 'Norveški A1', 3, 6, 1, 1, 7),
(0, 'Japanski A1', 2, 5, 1, 1, 10);
```

Slika 17. DML naredbe za tabelu KURS

Popunjavanje tabele REZERVACIJA:

```
INSERT INTO RezervacijaKursa.Rezervacija (datumRez, kursID, kandidatID) VALUES  
( '2026-05-01' , 1, 1),  
( '2026-05-01' , 1, 2),  
( '2026-05-02' , 2, 3),  
( '2026-05-03' , 3, 4),  
( '2026-05-04' , 5, 5),  
( '2026-05-05' , 6, 6),  
( '2026-05-05' , 7, 7),  
( '2026-05-06' , 8, 8),  
( '2026-05-07' , 1, 9),  
( '2026-05-08' , 2, 10);
```

Slika 18. DML naredbe za tabelu REZERVACIJA

8. SQL upiti sa kombinovanjem podataka iz više tabela, upotrebom agregatnih funkcija, podupita i slično

Opis rada upita: Ovaj upit služi za generisanje izveštaja o kandidatima i kursevima koje su rezervisali, uz standardizaciju tekstualnih podataka i logičku podelu. Upit spaja tabele kandidata, rezervacija, kurseva i nivoa, a zatim primenjuje sledeće transformacije: imena i prezimena kandidata se uniformišu (imena se prikazuju svim velikim, a prezimena svim malim slovima) radi urednijeg prikaza, sa naziva kurseva se uklanjaju suvišne praznine. oznaci nivoa (npr. "A1", "B2") se dodaje prefiks kako bi korisniku bilo jasnije o čemu se radi (npr. "Nivo-A1"). Pomoću CASE izraza vrši se automatska klasifikacija kurseva na Osnovne, Srednje ili Napredne u zavisnosti od ključnih slova u njihovom nazivu.

```
SELECT UPPER(k.ime) AS ImeKandidata, LOWER(k.prezime) AS PrezimeKandidata, LTRIM(RTRIM(ku.naziv)) AS Kurs,
STUFF(n.oznaka, 1, 0, 'Nivo-') AS OznakaNivoa,
CASE
    WHEN ku.naziv LIKE '%A%' THEN 'Osnovni'
    WHEN ku.naziv LIKE '%B%' THEN 'Srednji'
    ELSE 'Napredni'
END AS TipKursa
FROM RezervacijaKursa.Kandidat k
JOIN RezervacijaKursa.Rezervacija r ON k.kandidatID = r.kandidatID
JOIN RezervacijaKursa.Kurs ku ON r.kursID = ku.kursID
JOIN RezervacijaKursa.Nivo n ON ku.nivoID = n.nivoID;
```

Slika 19. SQL upit 1

Opis rada upita: Svrha ovog upita je praćenje aktivnosti lista čekanja po kursevima kroz vremensku analizu rezervacija. Upit grupiše podatke po kursu, jeziku i nivou i broji ukupan broj kandidata na listi čekanja za svaki kurs, pronalazi datum prve i poslednje rezervacije kroz MIN i MAX, koristi DATEDIFF kako bi izračunao raspon u danima između prve i poslednje rezervacije na listi čekanja što pokazuje koliko dugo se lista čekanja popunjavala, a zatim kroz DATEADD dodaje 30 dana na datum poslednje rezervacije kako bi prikazao okvirni očekivani datum početka kursa.

```
SELECT UPPER(ku.naziv) AS Kurs, UPPER(j.naziv) AS Jezik, n.oznaka AS Nivo, COUNT(r.rezervacijaID) AS BrojNaListiCekanja,
MIN(r.datumRez) AS PrvaRezervacija, MAX(r.datumRez) AS PoslednjaRezervacija, DATEDIFF(DAY, MIN(r.datumRez),
MAX(r.datumRez)) AS RasponDana, CONVERT(VARCHAR(10), DATEADD(DAY, 30, MAX(r.datumRez)), 104) AS OcekivaniPocetak
FROM RezervacijaKursa.Kurs ku
JOIN RezervacijaKursa.Rezervacija r ON ku.kursID = r.kursID
JOIN RezervacijaKursa.Jezik j ON ku.jezikID = j.jezikID
JOIN RezervacijaKursa.Nivo n ON ku.nivoID = n.nivoID
GROUP BY ku.naziv, j.naziv, n.oznaka;
```

Slika 20. SQL upit 2

Opis rada upita: Ovaj upit predstavlja analizu rezultata testiranja i izdvaja samo one kandidate koji su ostvarili dobre rezultate. Upit koristi podupit da izračuna prosečan broj poena na nivou cele škole, a zatim filtrira i prikazuje isključivo kandidate čiji je broj osvojenih poena veći od ukupnog proseka, vrši matematičku konverziju poena u decimalni zapis, koristi CASE logiku da poene automatski "prevede" u školske ocene i opise uspeha (Odličan, Vrlo dobar, Dovoljan, Nedovoljan).

```
SELECT k.ime, k.prezime, ra.osvojeniPoeni, (CAST(ra.osvojeniPoeni AS NVARCHAR(20)) + '%') AS ProcentualniUspeh,
CASE
    WHEN ra.osvojeniPoeni >= 90 THEN 'Odličan'
    WHEN ra.osvojeniPoeni >= 75 THEN 'Vrlo dobar'
    WHEN ra.osvojeniPoeni >= 50 THEN 'Dovoljan'
    ELSE 'Nedovoljan'
END AS EvaluacijaTesta
FROM RezervacijaKursa.Kandidat k
JOIN RezervacijaKursa.Radi ra ON k.kandidatID = ra.kandidatID
WHERE ra.osvojeniPoeni > (
    SELECT AVG(CAST(osvojeniPoeni AS NUMERIC(10,2))) FROM RezervacijaKursa.Radi
);
```

Slika 21. SQL upit 3

Opis rada upita: Ovaj upit vrši vremensko filtriranje podataka i fokusira se na tačno određeni period. Upit povezuje podatke o kandidatu, njegovoj rezervaciji, kursu koji je odabrao, kao i jeziku i intenzitetu tog kursa. Spaja ime i prezime u jedno polje ("Polaznik") radi lakšeg čitanja. Filtrira bazu tako što analizira datum rezervacije i propušta samo one zapise gde je mesec 5, a godina 2026.

```
SELECT k.ime + ' ' + k.prezime AS Polaznik, j.naziv AS Jezik, i.tip AS Intenzitet, r.datumRez
FROM RezervacijaKursa.Kandidat k
JOIN RezervacijaKursa.Rezervacija r ON k.kandidatID = r.kandidatID
JOIN RezervacijaKursa.Kurs ku ON r.kursID = ku.kursID
JOIN RezervacijaKursa.Jezik j ON ku.jezikID = j.jezikID
JOIN RezervacijaKursa.Intenzitet i ON ku.intenzitetID = i.intenzitetID
WHERE MONTH(r.datumRez) = 5 AND YEAR(r.datumRez) = 2026;
```

Slika 22. SQL upit 4

Opis rada upita: Ovaj upit predstavlja nadogradnju i složeniju verziju hronološkog pregleda za maj 2026. godine, sa akcentom na marketinšku i logističku analizu ponašanja polaznika. Upit obavlja sledeće operacije: cisti ime i prezime od skrivenih razmaka i bezbedno ih spaja u jednu celinu pomoću funkcije CONCAT, tip intenziteta kursa vizuelno standardizuje u mala slova, vrši konverziju sirovog datuma i vremena iz baze u prepoznatljivi evropski format sa tačkama (dan.mesec.godina) koristeći standard 104, koristi funkciju EOMONTH (End of Month) da uporedi datum prijave sa poslednjim danom u mesecu. Ako se polaznik upisao na samom kraju meseca, sistem ga označava kao "Last-Minute upis", u suprotnom dobija status "Regularan period upisa".

```

SELECT CONCAT(LTRIM(RTRIM(k.ime)), ' ', LTRIM(RTRIM(k.prezime))) AS Polaznik, j.naziv AS Jezik,
LOWER(i.tip) AS IntenzitetKursa, CONVERT(VARCHAR(10), r.datumRez, 104) AS FormatiranDatumRezervacije,
CASE
    WHEN CAST(r.datumRez AS DATE) = EOMONTH(r.datumRez) THEN 'Last-Minute upis'
    ELSE 'Regularan period upisa'
END AS StatusUpisa
FROM RezervacijaKursa.Kandidat k
JOIN RezervacijaKursa.Rezervacija r ON k.kandidatID = r.kandidatID
JOIN RezervacijaKursa.Kurs ku ON r.kursID = ku.kursID
JOIN RezervacijaKursa.Jezik j ON ku.jezikID = j.jezikID
JOIN RezervacijaKursa.Intenzitet i ON ku.intenzitetID = i.intenzitetID
WHERE MONTH(r.datumRez) = 5 AND YEAR(r.datumRez) = 2026;

```

Slika 23. SQL upit 5

9. Sekvence sa primerom njihove upotrebe

Opis rada koda: Ovaj deo projekta demonstrira kreiranje i primenu objekata baze podataka pod nazivom SEQUENCE (sekvence), koji služe kao automatski brojači za generisanje jedinstvenih identifikatora (primarnih ključeva). U kodu su kreirane dve zasebne sekvence (RezervacijaSeq i KandidatSeq). Obe su podešene da startuju od broja 1 i da se sa svakim novim pozivom automatski uvećavaju za jedan korak (INCREMENT BY 1).

** Primer upotrebe je pod tackom 6. "DDL naredbe za generisanje tabela", za tebele Rezervacija i Kandidat. **

```
IF OBJECT_ID('OdrzavanjeKursa.RezervacijaSeq', 'SO') IS NOT NULL
    DROP SEQUENCE OdrzavanjeKursa.RezervacijaSeq;
GO

CREATE SEQUENCE RezervacijaKursa.RezervacijaSeq
START WITH 1
INCREMENT BY 1;
go
```

Slika 24. Sekvenca za Rezervaciju

```
IF OBJECT_ID('OdrzavanjeKursa.KandidatSeq', 'SO') IS NOT NULL
    DROP SEQUENCE OdrzavanjeKursa.KandidatSeq;
GO

CREATE SEQUENCE RezervacijaKursa.KandidatSeq
START WITH 1
INCREMENT BY 1;
go
```

Slika 25. Sekvenca za Kandidata

10. Index

Opis rada koda: Ovaj deo fokusiran je na ubrzavanje rada sa podacima kroz kreiranje nekasterizovanog indeksa pod nazivom IDX_Kandidat_Prezime nad tabelom Kandidat. Indeks je primarno postavljen nad kolonom prezime, što znači da baza podataka sortira i organizuje kandidate prema njihovom prezimenu. Koristi se i klauzula INCLUDE koja u sam indeks bez dodatnog sortiranja ubacuje i podatke o imenu i email adresi kandidata.

```
CREATE NONCLUSTERED INDEX IDX_Kandidat_Prezime
ON RezervacijaKursa.Kandidat (prezime) INCLUDE (ime, email);
GO

SELECT prezime, ime, email
FROM RezervacijaKursa.Kandidat
WHERE prezime = 'Marković';
GO
```

Slika 26. Index

11. Procedure

Procedura `usp_EvidentirajTestIDodeliKurs` napravljena je kao procedura sa ulaznim parametrima i jednim `OUTPUT` parametrom kroz koji se pozivaocu vraća ID pronađenog kursa. Cela logika procedure je u `TRY/CATCH` blok koji hvata sve greške i kroz `THROW` baca jasne poruke sa custom kodovima, što omogućava precizno lociranje problema. Pre bilo kakvog upisa, procedura prolazi kroz tri validacije, proverava da li kandidat postoji, da li odabrani jezik postoji i da li su poeni u ispravnom opsegu od 0 do 100. Na osnovu ostvarenih poena, kroz `CASE` automatski se određuje oznaka nivoa, a zatim se iz tabele `Nivo` povlači odgovarajući `nivoID`. Nakon toga se kroz dva `INSERT` upita evidentira ulazni test u tabeli `UlazniTest` i beleži da je kandidat radio taj test u tabeli `Radi`, pri čemu se kroz `SCOPE_IDENTITY()` hvata ID novokreiranog testa. Na kraju, kursor prolazi kroz aktivne kurseve za odabrani jezik i određeni nivo i vraća prvi pronađeni kroz `OUTPUT` parametar, a ukoliko takav kurs ne postoji, procedura o tome ispisuje odgovarajuću napomenu.

[illegible]

Slika 27. Procedura EvidentirajTestIDodeliKurs

Procedura `usp_OtkaziRezervacijeNeaktivnihKurseva` napravljena je kao procedura sa OUTPUT parametrom. Ova struktura omogućava da procedura ne samo izvrši brisanje već i vrati informaciju koliko je tačno kandidata skinuto sa liste čekanja, Procedura prima jedan opcionu OUTPUT parametar `@brojObrisanih` koji na kraju izvršavanja sadrži ukupan broj obrisanih stavki. Pre bilo kakvog brisanja, prolazi kroz dve validacije, prvo proverava da li uopšte postoje neaktivni kursevi u sistemu, a zatim da li neki od njih ima kandidate na listi čekanja. Ako uslov nije ispunjen, procedura ispisuje poruku i prekida izvršavanje kroz RETURN. Centralni deo procedure je kursor koji prolazi kroz sve neaktivne kurseve koji imaju bar jednog kandidata na listi čekanja, za svaki izvršava DELETE koji uklanja sve kandidate sa liste čekanja, a kroz `@@ROWCOUNT` beleži koliko ih je obrisano u toj iteraciji. Taj broj se sabira u `@brojObrisanih` kroz svaku iteraciju kursora kako bi na kraju bio dostupan pozivaocu kao tačan ukupan broj svih uklonjenih kandidata. Tokom izvršavanja, za svaki obrađeni kurs procedura kroz PRINT ispisuje naziv kursa, njegov ID i broj obrisanih kandidata, a na kraju i ukupan zbir svih obrisanih, što pruža jasan pregled svega što je procedura uradila u jednom pozivu.

```

--nagrada
IF OBJECT_ID('RezervacijaKursa.usp_OtkaziRezervacijeNeaktivnihKurseva', 'P') IS NOT NULL
    DROP PROC RezervacijaKursa.usp_OtkaziRezervacijeNeaktivnihKurseva;
GO

CREATE PROC RezervacijaKursa.usp_OtkaziRezervacijeNeaktivnihKurseva
    @brojObrisanih INT = 0 OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM RezervacijaKursa.Kurs WHERE aktivan = 0)
    BEGIN
        PRINT 'Nema neaktivnih kurseva, nista nije obrisano.';
        RETURN;
    END

    IF NOT EXISTS (
        SELECT 1 FROM RezervacijaKursa.Rezervacija r
        JOIN RezervacijaKursa.Kurs ku ON r.kursID = ku.kursID
        WHERE ku.aktivan = 0
    )
    BEGIN
        PRINT 'Nema rezervacija za neaktivne kurseve.';
        RETURN;
    END

    DECLARE @kursID INT;
    DECLARE @nazivKursa VARCHAR(20);
    DECLARE @obrisanoZaKurs INT;

    DECLARE kursor_neaktivni CURSOR FOR
        SELECT DISTINCT ku.kursID, ku.naziv
        FROM RezervacijaKursa.Kurs ku
        JOIN RezervacijaKursa.Rezervacija r ON ku.kursID = r.kursID
        WHERE ku.aktivan = 0;

    OPEN kursor_neaktivni;
    FETCH NEXT FROM kursor_neaktivni INTO @kursID, @nazivKursa;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        DELETE FROM RezervacijaKursa.Rezervacija
        WHERE kursID = @kursID;

        SET @obrisanoZaKurs = @@ROWCOUNT;
        SET @brojObrisanih = @brojObrisanih + @obrisanoZaKurs;

        PRINT 'Kurs: ' + ISNULL(@nazivKursa, 'Bez naziva') +
            ' (ID: ' + CAST(@kursID AS VARCHAR) + ') ' +
            ' | Obrisano kandidata sa liste cekanja: ' + CAST(@obrisanoZaKurs AS VARCHAR);

        FETCH NEXT FROM kursor_neaktivni INTO @kursID, @nazivKursa;
    END

    CLOSE kursor_neaktivni;
    DEALLOCATE kursor_neaktivni;

    PRINT '-----';
    PRINT 'Ukupno obrisanih sa liste cekanja: ' + CAST(@brojObrisanih AS VARCHAR);

END;
GO

```

Slika 28. Procedura `OtkaziRezervacijuNeaktivnihKurseva`

12. Funkcija

Funkcija `NaprednaAnalizaKandidata` napravljena je kao Multi-Statement Table-Valued funkcija. Ova struktura zahteva da se na samom početku tačno definiše tabela i sve njene kolone (`@StatistikaTablica`), koja se zatim unutar `BEGIN/END` bloka puni `INSERT INTO` komandom. Funkcija prima dva parametra — obavezan `@kriterijum_poena` koji filtrira samo kandidate koji su prešli traženu granicu na testu, i opcioni `@jezikID` koji omogućava dodatno filtriranje po jeziku, a ako se izostavi funkcija vraća kandidate za sve jezike. Unutar same funkcije nalazi se `SELECT` upit koji povezuje šest različitih tabela, kombinujući `INNER JOIN` za rezultate testova i `LEFT JOIN` za praćenje rezervacija, intenziteta i kurseva, kako se ne bi izgubili kandidati koji još uvek nisu na listi čekanja. Podaci se formatiraju kroz više funkcija — ime i prezime se čiste od razmaka i spajaju u jedno polje, datum se konvertuje u evropski format kroz `CONVERT` sa stilom 104, a kroz dve odvojene `CASE` logike poenima se automatski dodeljuje tačna oznaka nivoa od A1 do C2 i tekstualni rang kandidata. Pored toga, kroz dodatnu `CASE` kolonu `StatusNaListi` funkcija jasno pokazuje da li je kandidat trenutno na listi čekanja ili još uvek nije rezervisao mesto ni na jednom kursu.

```
CREATE FUNCTION RezervacijaKursa.NaprednaAnalizaKandidata (
    @kriterijum_poena INT,
    @jezikID INT = NULL
)
RETURNS @StatistikaTablica TABLE (
    KandidatID INT,
    Polaznik VARCHAR(45),
    KontaktEmail VARCHAR(25),
    Jezik VARCHAR(20),
    Intenzitet VARCHAR(20),
    OsvojeniPoeni INT,
    ProcenatUspeha VARCHAR(20),
    OdradjeniKurs VARCHAR(2),
    RangKandidata VARCHAR(10),
    StatusNaListi VARCHAR(20),
    RezervisaniKurs VARCHAR(20),
    DatumUpisa VARCHAR(10)
)
AS
BEGIN
    INSERT INTO @StatistikaTablica
    SELECT
        k.kandidatID,
        CONCAT(LTRIM(RTRIM(k.ime)), ' ', LTRIM(RTRIM(k.prezime))) AS Polaznik,
        ISNULL(k.email, 'Nema_email') AS KontaktEmail, j.naziv AS Jezik,
        ISNULL(i.tip, 'Nema_kursa') AS Intenzitet, ra.osvojeniPoeni AS OsvojeniPoeni,
        (CAST(ra.osvojeniPoeni AS NVARCHAR(20)) + '%') AS ProcenatUspeha,
        CASE
            WHEN ra.osvojeniPoeni BETWEEN 0 AND 15 THEN 'A1'
            WHEN ra.osvojeniPoeni BETWEEN 16 AND 30 THEN 'A2'
            WHEN ra.osvojeniPoeni BETWEEN 31 AND 45 THEN 'B1'
            WHEN ra.osvojeniPoeni BETWEEN 46 AND 60 THEN 'B1'
            WHEN ra.osvojeniPoeni BETWEEN 61 AND 75 THEN 'B2'
            WHEN ra.osvojeniPoeni BETWEEN 76 AND 90 THEN 'C1'
            ELSE 'C2'
        END AS OdradjeniNivo,
        CASE
            WHEN ra.osvojeniPoeni >= 90 THEN 'Izuzetan (Nivo C2/C1)'
            WHEN ra.osvojeniPoeni >= 75 THEN 'Prosecan (Nivo B2/B1)'
            WHEN ra.osvojeniPoeni >= 50 THEN 'Dovoljan (Nivo B1/A2)'
            ELSE 'Ispod proseka (Potreban A1/A2)'
        END AS RangKandidata,
        CASE
            WHEN r.rezervacijaID IS NOT NULL THEN 'Na listi cekanja'
            ELSE 'Nije na listi'
        END AS StatusNaListi,
        ISNULL(ku.naziv, 'Nema rezervacije') AS RezervisaniKurs,
        ISNULL(CONVERT(VARCHAR(10), r.datumRez, 104), 'Nije upisan') AS DatumUpisa
    FROM RezervacijaKursa.Kandidat k
    JOIN RezervacijaKursa.Kurs ku ON k.kandidatID = ku.kandidatID
    JOIN RezervacijaKursa.Olasnost ut ON ra.olasnTestID = ut.olasnTestID
    JOIN RezervacijaKursa.Jezik j ON ut.jezikID = j.jezikID
    LEFT JOIN RezervacijaKursa.Rezervacije r ON k.kandidatID = r.kandidatID
    LEFT JOIN RezervacijaKursa.Kurs ku ON r.kursID = ku.kursID
    LEFT JOIN RezervacijaKursa.Intenzitet i ON ku.intenzitetID = i.intenzitetID
    WHERE ra.osvojeniPoeni >= @kriterijum_poena
        AND (@jezikID IS NULL OR ut.jezikID = @jezikID);

    RETURN;
END;
GO
```

Slika 29. Funkcija `NaprednaAnalizaKandidata`

13. Trigeri

Triger `tr_ObrisiZastareluRezervaciju` napravljen je kao AFTER triger koji se automatski aktivira nakon svakog INSERT-a u tabelu `Radi`, odnosno kada se evidentira da je kandidat radio ulazni test. Cela logika omotana je u TRY/CATCH blok koji hvata greške i ispisuje odgovarajuću poruku. Iz inserted tabele se uzimaju podaci o kandidatu, novom nivou i jeziku testa, nakon čega kursor prolazi kroz sve postojeće rezervacije tog kandidata za isti jezik ali drugačiji nivo. Svaka takva rezervacija se briše jer više ne odgovara trenutnom znanju kandidata, a kroz @@ROWCOUNT se broji koliko ih je uklonjeno. Na kraju se ispisuje ukupan broj obrisanih zastarelih rezervacija, a ako ih nema ide odgovarajuća napomena.

```
CREATE TRIGGER RezervacijaMurea.tr_ObrisiZastareluRezervaciju
ON RezervacijaMurea.Radi
AFTER INSERT
AS
BEGIN
    IF @@ROWCOUNT = 0 RETURN;

    SET NOCOUNT ON;

    BEGIN TRY

        DECLARE @kandidatID INT;
        DECLARE @ulazniTestID INT;
        DECLARE @noviNivoID INT;
        DECLARE @jezikID INT;
        DECLARE @etariKursID INT;
        DECLARE @etariKursIV VARCHAR(20);
        DECLARE @obrisano INT = 0;

        SELECT
            @kandidatID = i.kandidatID,
            @ulazniTestID = i.ulazniTestID,
            @noviNivoID = ut.nivoID,
            @jezikID = ut.jezikID
        FROM inserted i
        JOIN RezervacijaMurea.UlazniTest ut ON i.ulazniTestID = ut.ulazniTestID;

        DECLARE kursor_rezervacije CURSOR FOR
            SELECT r.kursID, ku.naziv
            FROM RezervacijaMurea.Rezervacija r
            JOIN RezervacijaMurea.Kurs ku ON r.kursID = ku.kursID
            WHERE r.kandidatID = @kandidatID
            AND ku.jezikID = @jezikID
            AND ku.nivoID != @noviNivoID;

        OPEN kursor_rezervacije;
        FETCH NEXT FROM kursor_rezervacije INTO @etariKursID, @etariKursIV;

        WHILE @@FETCH_STATUS = 0
        BEGIN
            DELETE FROM RezervacijaMurea.Rezervacija
            WHERE kandidatID = @kandidatID
            AND kursID = @etariKursID;

            SET @obrisano = @obrisano + @@ROWCOUNT;

            PRINT 'Obrisana zastarela rezervacija | Kandidat ID: ' + CAST(@kandidatID AS VARCHAR) +
                ' | Kurs: ' + ISNULL(@etariKursIV, 'Bez naziva') +
                ' (kursID: ' + CAST(@etariKursID AS VARCHAR) + ')';

            FETCH NEXT FROM kursor_rezervacije INTO @etariKursID, @etariKursIV;
        END

        CLOSE kursor_rezervacije;
        DEALLOCATE kursor_rezervacije;

        IF @obrisano = 0
            PRINT 'Nema zastarelih rezervacija za kandidata ID: ' + CAST(@kandidatID AS VARCHAR);
        ELSE
            PRINT 'Ukupno obrisanih zastarelih rezervacija: ' + CAST(@obrisano AS VARCHAR);

    END TRY
    BEGIN CATCH
        PRINT 'GRESKA u triggeru: ' + ERROR_MESSAGE();
    END CATCH

END;
GO
```

Slika 30. Triger `ObrisiZastareluRezervaciju`

Triger tr_ValidacijaRezervacije napravljen je kao INSTEAD OF triger koji se aktivira umesto svakog INSERT-a u tabelu Rezervacija, odnosno umesto svakog pokušaja dodavanja kandidata na listu čekanja. Cela logika omotana je u TRY/CATCH blok. Pre nego što dozvoli upis, triger prolazi kroz dve provere, prvo proverava da li kandidat već čeka na nekom kursu istog jezika, jer je u sistemu dozvoljena samo jedna rezervacija po jeziku, a zatim proverava da li je kurs na koji kandidat pokušava da se doda uopšte aktivan. Ako bilo koji od ova dva uslova nije ispunjen, upis se odbija uz odgovarajuću poruku. Tek ako obe provere prođu, izvršava se stvarni INSERT u tabelu Rezervacija i kandidat se uspešno dodaje na listu čekanja.

```
CREATE TRIGGER RezervacijaKursa.tr_ValidacijaRezervacije
ON RezervacijaKursa.Rezervacija
INSTEAD OF INSERT
AS
BEGIN
    IF @@ROWCOUNT = 0 RETURN;

    SET NOCOUNT ON;

    BEGIN TRY

        IF EXISTS (
            SELECT 1
            FROM inserted i
            JOIN RezervacijaKursa.Rezervacija r ON i.kandidatID = r.kandidatID
            JOIN RezervacijaKursa.Kurs ku_novi ON i.kursID = ku_novi.kursID
            JOIN RezervacijaKursa.Kurs ku_stari ON r.kursID = ku_stari.kursID
            WHERE ku_novi.jezikID = ku_stari.jezikID
        )
        BEGIN
            PRINT 'GRESKA: Kandidat vec ima rezervaciju za kurs istog jezika.
                Dozvoljena je samo jedna rezervacija po jeziku.';
        END
    ELSE
        BEGIN
            IF EXISTS (
                SELECT 1 FROM inserted i
                JOIN RezervacijaKursa.Kurs ku ON i.kursID = ku.kursID
                WHERE ku.aktivan = 0
            )
            BEGIN
                PRINT 'GRESKA: Nije moguće rezervirati mesto na neaktivnom kursu.';
            END
        ELSE
            BEGIN
                INSERT INTO RezervacijaKursa.Rezervacija (rezervacijaID, datumRez, kursID, kandidatID)
                SELECT rezervacijaID, datumRez, kursID, kandidatID
                FROM inserted;

                PRINT 'USPEH: Kandidat je uspesno dodat na listu cekanja.';
            END
        END
    END TRY
    BEGIN CATCH
        PRINT 'GRESKA u triggeru: ' + ERROR_MESSAGE();
    END CATCH

END;
GO
```

Slika 31. Triger ValidacijaRezervacije

14. Upotreba kursora u nekom od objekata

Procedura IzvestajListeCekanja napravljena je kao procedura koja prima ID kursa i generiše pregled svih kandidata koji se trenutno nalaze na listi čekanja za taj kurs. Glavni deo procedure je kursor koji funkcioniše kao pokazivač koji prolazi kroz rezultate upita red po red, što je glavna razlika u odnosu na obični SELECT koji vraća sve rezultate odjednom. Kursor prikuplja sve kandidate sa liste čekanja za dati kurs kroz SELECT koji spaja četiri tabele, a zatim za svakog kandidata posebno, formira poruku sa rednim brojem, imenom i prezimenom, email adresom, jezikom kursa i datumom kada se kandidat stavio na listu čekanja. Na kraju procedure ispisuje se i ukupan broj kandidata koji čekaju.

```
CREATE PROC RezervacijaKursa.IzvestajListeCekanja
@kurs_id INT
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM RezervacijaKursa.Kurs WHERE kursID = @kurs_id)
    BEGIN
        PRINT 'Greška: Kurs sa ID-jem ' + CAST(@kurs_id AS VARCHAR) + ' ne postoji.';
        RETURN;
    END

    DECLARE @ime VARCHAR(20);
    DECLARE @prezime VARCHAR(20);
    DECLARE @email VARCHAR(25);
    DECLARE @jezik VARCHAR(20);
    DECLARE @datumBez DATE;
    DECLARE @rbr INT = 1;
    DECLARE @naziv_kursa VARCHAR(20);

    SELECT @naziv_kursa = LTRIM(RTRIM(ku.naziv))
    FROM RezervacijaKursa.Kurs ku
    WHERE ku.kursID = @kurs_id;

    DECLARE kurzor_lista CURSOR FOR
    SELECT
        LTRIM(RTRIM(k.ime)),
        LTRIM(RTRIM(k.prezime)),
        ISNULL(k.email, 'Nema email'),
        j.naziv,
        r.datumBez
    FROM RezervacijaKursa.Rezervacija r
    JOIN RezervacijaKursa.Kandidat k ON r.kandidatID = k.kandidatID
    JOIN RezervacijaKursa.Kurs ku ON r.kursID = ku.kursID
    JOIN RezervacijaKursa.Jezik j ON ku.jezikID = j.jezikID
    WHERE r.kursID = @kurs_id
    ORDER BY r.datumBez ASC;

    OPEN kurzor_lista;
    FETCH NEXT FROM kurzor_lista INTO @ime, @prezime, @email, @jezik, @datumBez;

    IF @@FETCH_STATUS = -1
    BEGIN
        PRINT 'Nema kandidata na listi čekanja za kurs: ' + ISNULL(@naziv_kursa, 'Nepoznat');
        CLOSE kurzor_lista;
        DEALLOCATE kurzor_lista;
        RETURN;
    END

    PRINT '-----';
    PRINT 'LISTA ČEKANJA ZA KURS: ' + UPPER(ISNULL(@naziv_kursa, 'Nepoznat'));
    PRINT '-----';

    WHILE @@FETCH_STATUS = 0
    BEGIN
        PRINT CAST(@rbr AS VARCHAR) + '. ' +
            @ime + ' ' + @prezime +
            ' | Email: ' + @email +
            ' | Jezik: ' + @jezik +
            ' | Na listi od: ' + CONVERT(VARCHAR(10), @datumBez, 100);

        SET @rbr = @rbr + 1;
        FETCH NEXT FROM kurzor_lista INTO @ime, @prezime, @email, @jezik, @datumBez;
    END;

    CLOSE kurzor_lista;
    DEALLOCATE kurzor_lista;

    PRINT '-----';
    PRINT 'Ukupno na listi čekanja: ' + CAST(@rbr - 1 AS VARCHAR);

END;
GO
```

Slika 32. Kursor

15. Upotreba transakcije

Procedura `usp_ObrisiKandidata` napravljena je kao stored procedura sa jednim ulaznim parametrom i jednim opcionim parametrom `@potvrda` koji služi kao eksplicitna potvrda brisanja u slučaju da kandidat ima visoke rezultate. Pre pokretanja transakcije procedura prolazi kroz dve provere — prvo da li kandidat uopšte postoji u sistemu, a zatim da li ima evidentiran test sa 90 ili više poena, jer se takav kandidat smatra vrednim i ne može se obrisati bez eksplicitne potvrde kroz `@potvrda = 1`. Ove dve provere su namerno postavljene van transakcije jer nema smisla pokretati transakciju samo da bi se odmah stopirala. Nakon što provere prođu, kreira se privremena tabela `#Log` koja kroz celo izvršavanje beleži svaki korak brisanja sa opisom, brojem obrisanih redova i tačnim vremenom izvršavanja. Unutar `BEGIN TRANSACTION` bloka izvršavaju se tri `DELETE` operacije redom — prvo se brišu sve kandidatove rezervacije sa liste čekanja, zatim sva evidencija testova iz tabele `Radi`, i na kraju sam kandidat iz tabele `Kandidat`. Svaki korak se odmah upisuje u `#Log`. Ako sva tri koraka prođu bez greške, poziva se `COMMIT` koji trajno čuva sve izmene i ispisuje kompletan log kao izveštaj. Ako u bilo kom trenutku dođe do greške, `CATCH` blok poništava sve kroz `ROLLBACK`, grešku upisuje u log i ispisuje je korisniku, čime se garantuje da baza nikad ne ostane u delimično obrisanom stanju.

```
CREATE PROC usp_ObrisiKandidata
    @kandidatID INT,
    @potvrda INT = 0
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM Reservacija.Kandidat WHERE kandidatID = @kandidatID)
    BEGIN
        PRINT 'Greška: Kandidat sa ID ' + CAST(@kandidatID AS VARCHAR) + ' ne postoji.';
        RETURN;
    END

    DECLARE @kandidatID VARCHAR(20);
    DECLARE @reservacijaID VARCHAR(20);
    DECLARE @testID INT;
    DECLARE @testPoena INT;
    DECLARE @testVreme INT;

    SELECT
        @kandidatID = ID,
        @reservacijaID = rezervacijaID,
        @testID = testID,
        @testPoena = testPoena,
        @testVreme = testVreme
    FROM Reservacija.Kandidat
    WHERE kandidatID = @kandidatID;

    -- Proveravamo da li kandidat ima test sa 90+ poena
    SELECT @testPoena = testPoena FROM Radi
    WHERE kandidatID = @kandidatID;

    IF @testPoena < 90 AND @potvrda = 0
    BEGIN
        PRINT 'POVRATA: Kandidat ' + @kandidatID + ' sa ' + CAST(@testPoena AS VARCHAR) + ' poena (90+).';
        PRINT 'Dva uslova od tri za brisanje kandidata, preostaje provera sa @potvrda = 1.';
        RETURN;
    END

    CREATE TABLE #Log (
        ID INT,
        Opis VARCHAR(200),
        RezPoena INT,
        Vreme DATETIME DEFAULT GETUTIME()
    );

    BEGIN TRANSACTION;

    BEGIN TRY
        DELETE FROM Reservacija.Reservacija
        WHERE kandidatID = @kandidatID;

        SET @testID = @reservacijaID;

        DELETE FROM Radi
        WHERE kandidatID = @kandidatID;

        SET @testID = @reservacijaID;

        DELETE FROM Radi
        WHERE kandidatID = @kandidatID;

        SET @testID = @reservacijaID;

        INSERT INTO #Log (ID, Opis, RezPoena, Vreme)
        VALUES (1, 'Brisanje kandidata sa ID ' + CAST(@kandidatID AS VARCHAR) + ' i svih rezervacija i testova (ID: ' + CAST(@testID AS VARCHAR) + ')', @testPoena, GETUTIME());

        COMMIT TRANSACTION;

        PRINT 'Transakcija uspešno izvršena sa kandidatom ' + @kandidatID + ' sa ' + CAST(@testPoena AS VARCHAR) + ' poena.';
        PRINT 'Rezultati su upisani u tabelu #Log.';
        SELECT * FROM #Log ORDER BY ID;
    END TRY
    BEGIN CATCH
        IF @@ERROR > 0
            ROLLBACK TRANSACTION;

        INSERT INTO #Log (ID, Opis, RezPoena, Vreme)
        VALUES (2, 'Greška: Neuspešno izvršavanje transakcije (ID: ' + CAST(@testID AS VARCHAR) + ')', @testPoena, GETUTIME());

        PRINT 'Greška: ' + ERROR_MESSAGE();
        PRINT 'Transakcija poništena, sve promene su obrisane.';
        SELECT * FROM #Log ORDER BY ID;
    END CATCH

    DROP TABLE #Log;
END;
```

Slika 33. Transakcija

16. Zaključak

U okviru ovog projektnog zadatka projektovana je i implementirana baza podataka koja pokriva deo poslovnih procesa škole stranih jezika, konkretno podsistem za rezervaciju kurseva. Rad je započet opisom realnog sistema i konceptualnim modelovanjem celokupne baze kroz EER dijagram, nakon čega je odabrana i detaljno opisana podšema Rezervacija kursa. Za odabranu podšemu definisan je tabelarni prikaz svih obeležja i ograničenja, kao i relacioni model koji jasno prikazuje veze između entiteta.

Na osnovu toga kreirane su tabele kroz DDL naredbe sa svim potrebnim ograničenjima, a zatim su popunjene testnim podacima kroz DML naredbe. Napisano je pet SQL upita koji kombinuju podatke iz više tabela, koriste agregatne funkcije, podupite i različite transformacije podataka. Za automatsko generisanje ključeva implementirane su dve sekvence, a kreiran je i indeks nad tabelom Kandidat radi ubrzavanja pretrage po prezimenu.

Poseban akcenat stavljen je na implementaciju naprednih objekata baze podataka. Kreirane su dve procedure koje automatizuju evidenciju testova i otkazivanje rezervacija, table-valued funkcija za napredniju analizu kandidata po broju ostvarenih poena, kao i dva trigera koji štite integritet podataka pri unosu i izmeni rezervacija. Kroz proceduru sa dva kursora pokazano je kako se može prolaziti kroz podatke red po red i za svakog kandidata posebno donositi odluka, što standardnim upitima nije moguće. Na kraju je kroz transakcionu proceduru pokazano kako se više operacija može grupisati u jednu celinu, ako bilo šta pođe po zlu, sve se automatski poništava i baza ostaje u ispravnom stanju, kao da se ništa nije ni dogodilo.

Projekat je u celini pokazao kako dobro projektovana baza podataka kroz svoje objekte aktivno učestvuje u zaštiti, obradi i automatizaciji poslovnih procesa, a ne služi samo kao pasivno skladište podataka.